Cesar Moreno Fernandes – 3º AMS

Relatório – Geolocalização em Aplicativo React Native

1. Funcionamento no Projeto

A geolocalização do aplicativo foi implementada principalmente com a biblioteca expo-location. Ela é utilizada para solicitar autorização, obter uma posição pontual e acompanhar mudanças na posição do aparelho. O fluxo está dividido entre as telas do aplicativo, Hooks reutilizáveis e um serviço responsável pelo rastreamento em segundo plano.

app/rota/nova.tsx: na criação de uma rota, a função captureOrigin() chama Location.requestForegroundPermissionsAsync() para solicitar acesso à localização enquanto o aplicativo está em uso. Se a permissão for concedida, chama Location.getCurrentPositionAsync() para obter uma única posição e preencher as coordenadas de início da rota.

app/(tabs)/rastrear.tsx: é a tela principal de registro do percurso. A função start() verifica se a permissão de primeiro plano já foi concedida e, se necessário, solicita a autorização. Em seguida, usa getCurrentPositionAsync() para registrar o ponto inicial e
Location.watchPositionAsync() para receber novas posições durante o deslocamento. O aplicativo atualiza o mapa, calcula a distância e envia os pontos para armazenamento local. A função stopWatching() remove a inscrição de monitoramento quando o rastreamento termina.

A tela também utiliza useCompass(true) para acompanhar a direção indicada pela bússola e usePedometer(tracking) para contar os passos durante o percurso. Esses recursos complementam o rastreamento, mas não são responsáveis por obter as coordenadas GPS.

src/hooks/usePermissions.ts: contém o Hook useLocationPermissions(), que consulta o estado das permissões de primeiro e segundo plano por meio de getForegroundPermissionsAsync() e getBackgroundPermissionsAsync(). Também disponibiliza requestForeground() e requestBackground(), que solicitam essas permissões e atualizam o estado. O mesmo arquivo possui useNotificationPermission(), voltado às notificações, não ao GPS.

app/(tabs)/ajustes.tsx: utiliza esses Hooks para mostrar se a localização em primeiro e segundo plano está liberada. A opção “Liberar localização em segundo plano” chama a função que solicita essa autorização. A tela também solicita permissão para notificações quando o usuário ativa o lembrete diário.

src/hooks/useCompass.ts: usa Location.watchHeadingAsync() para receber atualizações da direção do aparelho. O Hook guarda os valores com useState, inicia o acompanhamento com useEffect e remove a inscrição ao ser desativado ou desmontado.

src/hooks/usePedometer.ts: usa Pedometer.isAvailableAsync() e Pedometer.watchStepCount(), da biblioteca expo-sensors, para verificar a disponibilidade do sensor e acompanhar a contagem de passos. Também utiliza useEffect e useState para controlar a leitura.

src/services/locationTask.ts: é o serviço que define a tarefa de localização em segundo plano com expo-task-manager. A função
TaskManager.defineTask() registra a tarefa, e persistPoint() grava as coordenadas e outros dados do ponto no banco local.
buildBackgroundTaskOptions() configura precisão, intervalo de tempo e distância. O código da tela de rastreamento contém chamadas para iniciar e parar essa tarefa, mas a constante USE_BACKGROUND_TASK está definida como false; portanto, nesse arquivo, o rastreamento em segundo plano não é ativado atualmente.

2. Pacotes e Dependências Identificados

Pelos imports dos arquivos analisados, as principais bibliotecas relacionadas são:

- expo-location: solicita permissões, obtém a posição atual, acompanhamudanças de localização e fornece atualizações da bússola.
- expo-task-manager: registra e executa a tarefa que pode receberatualizações de localização em segundo plano.
- expo-sensors: fornece o sensor de passos usado pelo HookusePedometer.
- react-native-maps: exibe o mapa, a posição do usuário, os marcadores ea linha do percurso na tela de rastreamento.
- expo-image-picker: solicita acesso à biblioteca de mídia e permiteselecionar a foto de capa da rota.

Essas dependências são identificadas pelo uso nos arquivos enviados. Como o package.json não foi incluído, não é possível confirmar todas as dependências instaladas nem suas versões.

3. Permissões de Acesso

Para obter a localização enquanto o aplicativo está aberto, é necessário solicitar a permissão de localização em primeiro plano. No projeto, isso é feito com Location.requestForegroundPermissionsAsync(). Para continuar recebendo atualizações quando o aplicativo estiver em segundo plano, é necessária também a permissão correspondente, solicitada por Location.requestBackgroundPermissionsAsync().

Além das solicitações feitas pelo código, o projeto precisa ter as configurações nativas adequadas para cada plataforma. No Android, isso envolve declarar as permissões de localização necessárias no manifesto/configuração do aplicativo. No iOS, é necessário informar ao sistema o motivo do acesso à localização e configurar o modo de localização em segundo plano quando essa função for utilizada. Os arquivos de

configuração nativa não foram enviados, então não foi possível verificar essas declarações.

A autorização em tempo de execução é obrigatória porque a localização é um dado pessoal e o sistema operacional permite que o usuário decida se o aplicativo pode acessá-la. O aplicativo deve explicar a finalidade, verificar o resultado da solicitação e tratar a recusa, sem presumir que a permissão já foi concedida.

O projeto também possui outras permissões, usadas em funções diferentes: galeria para selecionar a foto de capa (a solicitação aparece em nova.tsx); câmera, contatos e galeria nas opções relacionadas ao percurso, conforme o funcionamento informado do aplicativo; e notificações para o lembrete diário. Essas autorizações não substituem a permissão de localização. Nos arquivos enviados, a solicitação de galeria está visível, enquanto os detalhes de solicitação de câmera e contatos não aparecem integralmente.

4. Busca Pontual vs. Monitoramento

Busca pontual: obtém a posição do aparelho uma única vez, no momento da chamada. No projeto, isso acontece com Location.getCurrentPositionAsync(), usado para definir o ponto inicial de uma rota ou de um percurso. É adequado quando basta saber onde o usuário está naquele instante.

Monitoramento contínuo: mantém uma inscrição ativa e recebe novas posições à medida que o aparelho se desloca. No projeto, isso é feito com Location.watchPositionAsync(). É necessário para desenhar o caminho percorrido, atualizar a posição no mapa e calcular distância e velocidade ao longo do trajeto. Como o monitoramento continua solicitando atualizações, deve ser encerrado quando não for mais necessário, removendo a inscrição.

Em resumo, a busca pontual responde “onde estou agora?”, enquanto o monitoramento responde “por onde estou me deslocando?”. O monitoramento tende a consumir mais bateria e exige atenção especial às permissões e ao encerramento correto.

5. Outra Solução de Mercado

react-native-geolocation-service: é uma biblioteca alternativa para projetos React Native que precisam obter a posição atual ou acompanhar alterações de localização. Ela oferece métodos como getCurrentPosition() e watchPosition() e, no Android, utiliza o provedor de localização combinada do Google Play Services (FusedLocationProviderClient) para tentar melhorar a confiabilidade em comparação com o provedor Android padrão. É uma opção a considerar em projetos React Native com configuração nativa própria, especialmente quando se deseja essa integração com os serviços do Google. A biblioteca não elimina a necessidade de configurar e solicitar as permissões do sistema.

Neste projeto, expo-location atende às chamadas de localização já utilizadas e se integra ao ecossistema Expo, inclusive com a tarefa em segundo plano definida por expo-task-manager.

6. Referências

AGONTUK. react-native-geolocation-service – documentação e instalação. Disponível em: https://github.com/Agontuk/react-native-geolocation-service. Acesso em: 9 nov. 2026.

NPM. react-native-geolocation-service. Disponível em: https://www.npmjs.com/package/react-native-geolocation-service. Acesso em: 9 nov. 2026.